Test Plan for Borsibaar

1. Testing objectives

Ensure users can use the web application without issue. Verify that users can change data in the database via the web client, for example adding new drinks or changing the existing ones goes smoothly: tests fail if a value entered into a field is incorrect, for example words entered into price field, or fields left empty.

Confirm the docker cluster starts without issues. During testing, it is verified that containers, services, networking and orchestration features function correctly across the entire cluster, support scalability, and ensure a reliable and fault-tolerant system.

2. Testing levels

Unit testing - Singular method or component tests in isolation
Integration testing - Verify with tests that different components of the application work together. For example, data is stored and changed in the database (API). Or checking whether the frontend and backend can communicate, or backend and database.
System integration testing - Verify that the whole system is still running as it should. Parts of this are docker container health, verifying that running commands don’t shut down any services and so on.

3. Test scope

Adding new drinks

1) Verify tests fail if the "Add Drink" button doesn't open modal with form fields.

2) Verify tests fail if required field is left empty, too long or wrong data type.

   Fields (`frontend/app/protected/sidebar/inventory/page.tsx`):

   `name`: required text, max length (max 30 chars).
   `description`: optional text (max 200 chars).
   `categoryId`: required, valid existing category id.
   `currentPrice`: required, numeric (double), ≥ 0.
   `minPrice`: required, numeric (double), ≥ 0 and ≤ `currentPrice`.
   `maxPrice`: required, numeric (double), ≥ `currentPrice`.
   `initialQuantity`: required, numeric (double), ≥ 0.
   `notes`: optional text.

3) Updates are saved in the database, otherwise tests fail.

Editing existing drinks:

1) Verify tests fail if the "Edit Drink" button doesn't open modal with form fields.
2) Same validation rules as in Adding new drinks section, except there is no `initialQuantity` and `notes` field.
3) Updates are saved in the database, otherwise tests fail.

Docker environment

1) `docker compose up` completes without errors.

2) All services are running: `docker compose ps` shows PostgreSQL, backend, frontend are all up.
3) Backend: /actuator/health returns “UP” (logic in nginx/conf.d/borsibaar-https.conf)
4) frontend loads at expected URL http://localhost:3000.
5) PostgreSQL running and accepting connections docker compose exec postgres pg_isready -U postgres.

4. Test approach

4a. Back-end testing:
Junit and Mockito are used to test backend components and to validate REST API endpoints.

4b. Front-end testing:
Jest Unit testing is used to validate input data and to test user interface logic.

4c. System integration testing:
Reliability and fault-tolerance testing is performed in Docker, using health checks and verifying automatic service recovery

4d. Security testing: Authentication, access control.

4e. Performance testing: System behavior under high load is evaluated.

5. Test environment

Browsers - Chrome, Safari, Firefox both on Desktop and Mobile.

Smoke Test - Verify the Docker-based development environment starts successfully (e.g. `docker compose up` finishes, all services are healthy, backend and frontend are reachable).

6. Entry and exit criteria

Entry

- Code is pushed to Git
- `Docker compose up` runs successfully
- All services are healthy

Exit

- All critical tests pass
- No critical bugs remain
- Test report is created
- Bugs are listed (if found)

7. Roles and responsibilities

Developers are responsible for executing, automating, and fixing defects related to unit and integration tests for the source code they implement.

The QA team performs system and system integration testing, documents identified defects, and confirms system readiness prior to deployment.

Tests are executed automatically within the CI/CD pipeline, and test results are documented in test reports.

8. Risks and assumptions

Assumptions:

The test environment and required servers are available.

All dependencies (e.g., databases, third-party services) are functioning as expected.

The team is sufficiently trained or familiar with the system's operation.

Necessary access and permissions (API keys, authentications) are in place.

Test data is complete and realistic.

Risks:

Interruptions or maintenance periods of third-party services (e.g., Google O Auth).

Potential security or data protection issues (e.g., personal data leaks).

Technical problems in the test environment or on servers.

Unclear or changing requirements that may affect the scope of testing.

Schedule delays or lack of team resources.

9. Test deliverables

Test Plan Summary

Field	**Details**
Scope	Product management (add/edit drinks)

Objective	Verify user interactions work reliably with proper validation
Test Strategy	Unit → Integration → System tests
Schedule	1 week

Test Cases (9 total)

Test: Add new drink (3 cases):

1. "New Product" button opens modal with form fields
   Expected result: The modal opens, all form fields are displayed and editable.
2. Invalid input shows error message, data not saved
   Expected result: A proper error message is displayed, the data does not appear in the list and is not saved to the database.
3. Valid input → "Create Product" saves to DB, appears in list
   Expected result: Data is saved to the database, and the new product is immediately displayed in the list.

Test: Edit existing drink (3 cases):

4. "Edit" button opens modal with pre-filled data

   Expected result: Modal opens, all fields are pre-filled with the current product data and are editable.

5. Invalid input shows error message, original data unchanged

Expected result: A proper error message is displayed, and the original data remains unchanged in the database and list.

6. Valid changes → "Save" updates DB, visible after refresh

Expected result: Changes are saved to the database, and the updated product is visible in the list after page refresh.

Test: Validation (3 cases):

7. Price edge cases (negative, min>current, max<current)

Expected result: Validation error is displayed, and the data is not saved to the database.

8. Required fields empty → blocked with error

Expected result: Required field errors are shown, and the form submission is blocked.

9. Text limits exceeded → blocked with error

Expected result: A validation error is displayed indicating character limit exceeded, and the data is not saved.

Test Data

Type	**Valid**	**Invalid**
name	"Monster Energy Zero Sugar"	"No", "" (empty), 50+ chars

Type	Valid	Invalid
categoryId	Existing category from dropdown	Invalid/non-existent ID
currentPrice	"2.00"	"-1", "abc", ""
minPrice	"1.00"	"3.00" (> current), ""
maxPrice	"10.00"	"1.00" (< current), ""
initialQuantity	"5.50"	"-2", "abc", ""

Valid data:

- Product name: Monster Energy Zero Sugar
- *Category*: Non-alcoholic
- Price: *2*
- Minimum price: 1
- Maximum price: 10
- Product name length: within allowed character limit

Invalid data:

- Product name: No
- Price: 0
- Maximum price: 15000

Empty required fields:

- Product name: *empty*
- Category: empty

Text limit exceeded:

- Product name: string exceeding the allowed character limit

Expected Reports (Test Output Report)

- The modal window opens correctly.
- Validation errors are displayed for invalid input.
- No database changes occur when invalid input is provided.
- A new record is created in the database when valid input is submitted.
- The new product is immediately displayed in the product list.
- Form submission is blocked when validation fails.
- No invalid data is saved to the database.
- Users receive clear feedback indicating which fields contain errors.

Expected Reports

- Test execution log: pass/fail per case, screenshots of errors
- Defect list: bugs found, severity, repro steps
- Backend coverage: ProductService → target 90%
- Test summary: # passed/failed, coverage %, open issues